

sen-TapIn — Wiring & Setup Instructions

NFC Access & Attendance Terminal — Raspberry Pi Pico 2 W + DS3231 + DFRobot NFC Module V2.0 (PN532) + 0.96" IIC OLED (SSD1306) + MicroSD + WS2811 — button-less auto-toggle build

Product version	v0.1.0 (baseline) — MicroPython build (variant: +mp), auto-toggle flow per sen-TapIn.png
Pin map source	HDD-003 Rev 1.0 §3, with one deviation: the IN/OUT buttons (GP13/GP14) are not fitted — see §2. Record as an HDD-003 revision.
Related documents	REQ-SPEC-003 (requirements), CS-003 (coding standard), TP-003 (test plan), sen-TapIn.png (process flow)
Firmware	sen-TapIn/ folder — board.py, config.py, rtc.py, nfc.py, sd_log.py, ui.py, access.py, main.py, lib/
Date	31 July 2026

1. What you need

Item	Role	Interface
Raspberry Pi Pico 2 W (RP2350)	Central controller	—
DFRobot Gravity UART & I2C NFC Module V2.0 (DFR0231-H, PN532)	NFC card reader, 13.56 MHz	I2C @ 0x24
DS3231 RTC module (+ CR2032 coin cell)	Battery-backed timekeeping	I2C @ 0x68
0.96" IIC OLED 128×64 (SSD1306)	User display	I2C @ 0x3C
MicroSD card module (SPI, with on-board regulator/level shifter) + card	Allow-list + attendance log	SPI0
WS2811 addressable RGB LED breakout	Status colour	Single-wire (GP12)
Passive buzzer	Grant / deny tones	PWM (GP15)
2-channel relay board (only IN1 used) + low-voltage indicator LED	“Lock” indicator	GPIO (GP10)
Breadboard, jumper wires, micro-USB cable	Assembly / power	—
<i>Recommended:</i> 74AHCT125 level shifter	WS2811 data-line level (REQ-HW-004)	—

Tools: a computer with Thonny (or mpremote), and one or more ISO 14443A test cards (MIFARE Classic / NTAG).

2. Read this first — safety & constraints

Relay = indicator LED only (REQ-S-001 / HDD REQ-HW-003).

Do not connect mains voltage, a solenoid, a door strike, a motor, or any inductive load to the relay in v0.1.0. The relay channel switches a low-voltage indicator LED only. A real lock needs its own supply, a flyback diode, and a revision of HDD-003.

UID authorisation is not security (REQ-S-002 / HDD REQ-HW-002).

Card UIDs are trivially cloneable. sen-TapIn v0.1.0 is a learning and attendance baseline, not a security device.

Build deviation — no IN/OUT buttons; direction auto-toggles.

This build omits the two mode buttons of HDD-003 §3.3 / REQ-F-002. The terminal is always listening: tap a card any time. Each authorised card alternates direction automatically (first granted tap = IN, next = OUT, ...), rebuilt from the SD log at start-up so it survives power cycles. The same card is then ignored for

10 s

after a decided tap, so one presentation cannot double-toggle. GP13/GP14 are unallocated. This deviates from REQ-F-002/003/012 and HDD-003 Rev 1.0 — record it as a revision of both documents (Lead Engineer approval).

The rest of the pin map stays frozen (HDD REQ-HW-001).

Wire exactly as in Section 3. The MicroPython and later C builds both rely on this wiring; any change requires a revision of HDD-003 (and then one edit in `board.py` only).

Stay inside the USB current budget (HDD REQ-HW-005).

The SD module, relay board and WS2811 all share the 5 V VBUS rail from USB. Keep combined peak draw within what your USB port supplies; never power a real lock from VBUS.

Common ground is mandatory (HDD-003 §3.4).

Every module — 3.3 V powered and 5 V powered alike — must share GND with the Pico.

3. Master pin map (frozen — HDD-003 §3)

Pico 2 W GPIO	Physical pin	Function	Connects to
GP4	6	I2C0 SDA	DS3231 SDA + SSD1306 SDA + PN532 SDA (shared)
GP5	7	I2C0 SCL	DS3231 SCL + SSD1306 SCL + PN532 SCL (shared)
GP10	14	Digital out	Relay board IN1 (“lock” indicator)
GP11	15	Digital out	Relay board IN2 (spare — reserved, parked off)
GP12	16	Timed serial	WS2811 DIN (via level shifter — see §4.5)
GP13 / GP14	17 / 19	—	<i>Unallocated in this build (were IN/OUT buttons in HDD-003 Rev 1.0). Leave unconnected.</i>
GP15	20	PWM out	Passive buzzer (+)
GP16	21	SPI0 MISO	MicroSD DO / MISO
GP17	22	SPI0 CS	MicroSD CS
GP18	24	SPI0 SCK	MicroSD CLK / SCK
GP19	25	SPI0 MOSI	MicroSD DI / MOSI
3V3 OUT	36	3.3 V rail	DS3231 VCC, SSD1306 VCC, PN532 VCC
VBUS	40	5 V rail (USB)	SD module VCC, relay board VCC, WS2811 5V, level-shifter VCC
GND	3, 8, 13, 18, 23, 28, 33, 38	Ground	All modules (single common ground)

Pico 2 W — used pins at a glance (top view, USB at top)

```

+-----[ USB ]-----+
      GP0  1|
      GP1  2|
GND rail <-- GND  3|
      GP2  4|
      GP3  5|
I2C0 SDA (bus) <-- GP4  6|
I2C0 SCL (bus) <-- GP5  7|
      GND  8|
      GP6  9|
      GP7 10|
      GP8 11|
      GP9 12|
      GND 13|
Relay IN1 <-- GP10 14|
Relay IN2 <-- GP11 15|
WS2811 DIN <-- GP12 16|
(unused)      GP13 17|
      GND 18|
(unused)      GP14 19|
Buzzer (+) <-- GP15 20|

      Raspberry Pi
      Pico 2 W
      (top view,
      USB at top)

      40 VBUS --> 5V rail (SD, relay, WS2811)
      39 VSYS
      38 GND --> GND rail
      37 3V3_EN
      36 3V3 --> 3.3V rail (RTC, OLED, NFC)
      35 ADC_VREF
      34 GP28
      33 GND
      32 GP27
      31 GP26
      30 RUN
      29 GP22
      28 GND
      27 GP21
      26 GP20
      25 GP19 --> SD MOSI (DI)
      24 GP18 --> SD SCK (CLK)
      23 GND
      22 GP17 --> SD CS
      21 GP16 --> SD MISO (DO)

```

4. Wiring, module by module

Power everything down (USB unplugged) while wiring. Suggested order below; each step is independently checkable.

4.0 Step 0 — rails

- Run a **GND rail** on the breadboard from any Pico GND pin (e.g. physical pin 3 or 38).
- Run a **3.3 V rail** from Pico **3V3 OUT (pin 36)** — feeds the I2C trio only.
- Run a **5 V rail** from Pico **VBUS (pin 40)** — feeds SD module, relay board, WS2811.

4.1 Shared I2C bus — DS3231 + SSD1306 + PN532

All three devices sit on the same two wires. Chain SDA to SDA to SDA, SCL to SCL to SCL.

Module pin	DS3231 (0x68)	0.96" IIC OLED (0x3C)	NFC Module V2.0 (0x24)	Pico connection
VCC	VCC	VCC	VCC (+)	3.3 V rail (3V3, pin 36)
GND	GND	GND	GND (–)	GND rail
SDA	SDA	SDA	SDA (marked D/T on some revisions)	GP4 (pin 6)
SCL	SCL	SCL	SCL (marked C/R on some revisions)	GP5 (pin 7)

NFC Module V2.0 mode selector: set it to I2C before first power-up.

The DFRobot Gravity UART & I2C NFC Module V2.0 (DFR0231-H) speaks UART or I2C, chosen by its on-board switch. HDD-003 §2.2 requires I2C mode. In UART mode the firmware will report “NFC not found”.

Pull-ups (HDD-003 §3.1):

the breakout boards carry their own SDA/SCL pull-up resistors — do not add external ones. If the bus misbehaves with all three connected, one module’s pull-ups can be removed to reduce stacking.

Fit the CR2032 coin cell in the DS3231 so time survives power cycles (REQ-NF-004).

4.2 MicroSD module — dedicated SPI0 bus

SD module pin	Pico connection
VCC	5 V rail (VBUS, pin 40) — module has its own 3.3 V regulator + level shifting (HDD-003 §2.5)
GND	GND rail
CLK / SCK	GP18 (pin 24)
DI / MOSI	GP19 (pin 25)
DO / MISO	GP16 (pin 21)

CS	GP17 (pin 22)
----	---------------

If your SD breakout has no regulator

(bare socket board), power it from 3.3 V instead — check your specific board before applying 5 V.

4.3 Relay board (“lock” indicator)

Relay board pin	Pico connection
VCC	5 V rail (VBUS, pin 40)
GND	GND rail
IN1	GP10 (pin 14)
IN2	GP11 (pin 15) — spare, held inactive by firmware

Indicator load on channel 1: wire a low-voltage LED (+ series resistor, e.g. 330 Ω) through the relay contacts: 3.3 V rail → **COM**; **NO** (normally open) → resistor → LED → GND. Using NO means the LED is off unless the firmware deliberately pulses the relay — the fail-closed state (REQ-S-003).

Active-low boards:

many opto-isolated relay boards energise when IN1 is pulled LOW. If the relay clicks ON at boot, set `RELAY_ACTIVE_HIGH = False` in `config.py` — do not rewire.

4.4 Passive buzzer

Buzzer pin	Pico connection
+ (signal)	GP15 (pin 20)
–	GND rail

Must be a **passive** buzzer (needs a PWM tone). An active buzzer will only make one fixed note and cannot produce the distinct grant/deny tones of REQ-F-005.

4.5 WS2811 RGB LED

WS2811 pin	Pico connection
5V / VCC	5 V rail (VBUS, pin 40)
GND	GND rail
DIN (data in)	GP12 (pin 16) — via level shifter (recommended)

Data-line level must be verified during bring-up (HDD REQ-HW-004).

The WS2811 is a 5 V part; the Pico drives 3.3 V logic. A single LED at close range often works directly, but it is not guaranteed. Robust options: (a) a 74AHCT125 buffer — GP12 → 1A, 1Y → DIN, VCC=5 V, GND common, 1OE → GND; or (b) power the module at a lower supply voltage if the breakout permits. If colours are wrong/flickery without a shifter, fit one.

If red and green appear swapped, your LED is GRB-order: set `WS2811_ORDER = "GRB"` in `config.py`. If it is a 400 kHz part, set `WS2811_TIMING = 0`.

5. SD card preparation

1. Format a MicroSD card as **FAT32** (HDD-003 §3.2).
2. Copy `sen-TapIn/sd-card/allowlist.txt` to the **root** of the card.
3. Add one line per authorised card: `UID,Name` — e.g. `04A2B3C4,R. Madanayaka`. UID is uppercase hex, no separators; names must not contain commas. Lines starting with `#` are ignored (REQ-F-008).
4. Insert the card into the SD module. The log file `attendance.csv` is created automatically on the first event, with header `timestamp,uid,name,direction,result` (REQ-SPEC-003 §4).

Tip: to learn a card's UID, run the terminal and tap the card — the Denied event is logged to `attendance.csv` with its raw UID (REQ-F-010). Copy that UID into `allowlist.txt`; the allow-list is re-read at every decision, so the change applies without a restart.

6. Firmware installation

1. **Flash MicroPython:** download the latest **Pico 2 W** (RPI_PICO2_W) UF2 from micropython.org/download. Hold **BOOTSEL** while plugging in the USB cable, then drag the UF2 onto the RP2350 drive. The board reboots into MicroPython.
2. **Copy the firmware** from the `sen-TapIn/` folder to the board — all nine `.py` files plus the `lib/` folder:

```
cd ~/Desktop/NFC/sen-TapIn
pip install mpremote          # once
mpremote cp board.py config.py rtc.py nfc.py sd_log.py ui.py \
    access.py main.py :
mpremote mkdir lib
mpremote cp lib/ssd1306.py lib/sdcard.py :lib/
```

(Or in Thonny: View → Files, select the files, right-click → “Upload to /”, with the two drivers uploaded into `/lib`.)

3. **Reset** the board (unplug/replug or Ctrl-D in the REPL). `main.py` runs automatically at power-up.

7. Set the real-time clock (once)

With the wiring done and firmware copied, open a REPL (Thonny, or `mpremote repl`), interrupt the program with **Ctrl-C**, and set the DS3231 to the current local time:

```
from machine import I2C, Pin
import board, rtc
i2c = I2C(board.I2C_BUS_ID, sda=Pin(board.PIN_I2C_SDA),
          scl=Pin(board.PIN_I2C_SCL), freq=board.I2C_FREQ_HZ)
rtc.rtc_init(i2c)
rtc.rtc_set(2026, 7, 30, 9, 41, 0)    # year, month, day, hour, minute, second
rtc.rtc_now()                        # verify -> (True, '2026-07-30T09:41:0x')
```

The coin cell keeps this time across power cycles (REQ-NF-004). Press Ctrl-D to restart the terminal.

8. First bring-up checklist

8.1 I2C scan — must find exactly three devices (HDD-003 §3.1)

```
from machine import I2C, Pin
import board
i2c = I2C(board.I2C_BUS_ID, sda=Pin(board.PIN_I2C_SDA),
          scl=Pin(board.PIN_I2C_SCL), freq=board.I2C_FREQ_HZ)
[hex(a) for a in i2c.scan()]      # expect ['0x24', '0x3c', '0x68']
```

Address	Device	If missing, check
0x24	PN532 (DFR0231-H)	Mode switch on I2C? SDA/SCL swapped? Power on 3.3 V?
0x3C	SSD1306 OLED	Some clones use 0x3D — if so this is a hardware deviation; note it and update HDD-003, then board.py.
0x68	DS3231 RTC	VCC/GND orientation; module seated.

8.2 Functional pass

- Power up → OLED shows boot screen, then idle: time, date, “Tap your card” (REQ-F-009); RGB shows dim idle tint. The reader is already listening — no button press needed.
- Tap an **unknown** card → red + error tone + “DENIED”; relay stays off (REQ-F-010); one DENIED line (direction –) in attendance.csv.
- Add that UID to allowlist.txt, wait 10 s, tap again → green + confirmation tone + relay LED pulse + “GRANTED IN” with the name (REQ-F-005); GRANTED line with direction IN.
- Feedback appears within 1 s of the tap (REQ-NF-001).
- Re-tap the same card **within 10 s** → ignored: no feedback change, nothing logged, direction not toggled (same-card delay).
- Re-tap the same card **after 10 s** → “GRANTED OUT” — direction auto-toggled; next tap goes back to IN.
- Power-cycle, tap the card again after 10 s → the toggle continues from the last logged direction (state rebuilt from attendance.csv).
- Tap a **second** authorised card inside another card’s 10 s window → it is decided normally — the delay is per card.
- Remove the SD card, reboot → OLED shows FAULT, no card reads, relay off (REQ-NF-007, fail-closed). Re-insert; the terminal recovers on its 5 s retry.
- Power-cycle → clock still correct (battery backup, REQ-NF-004).

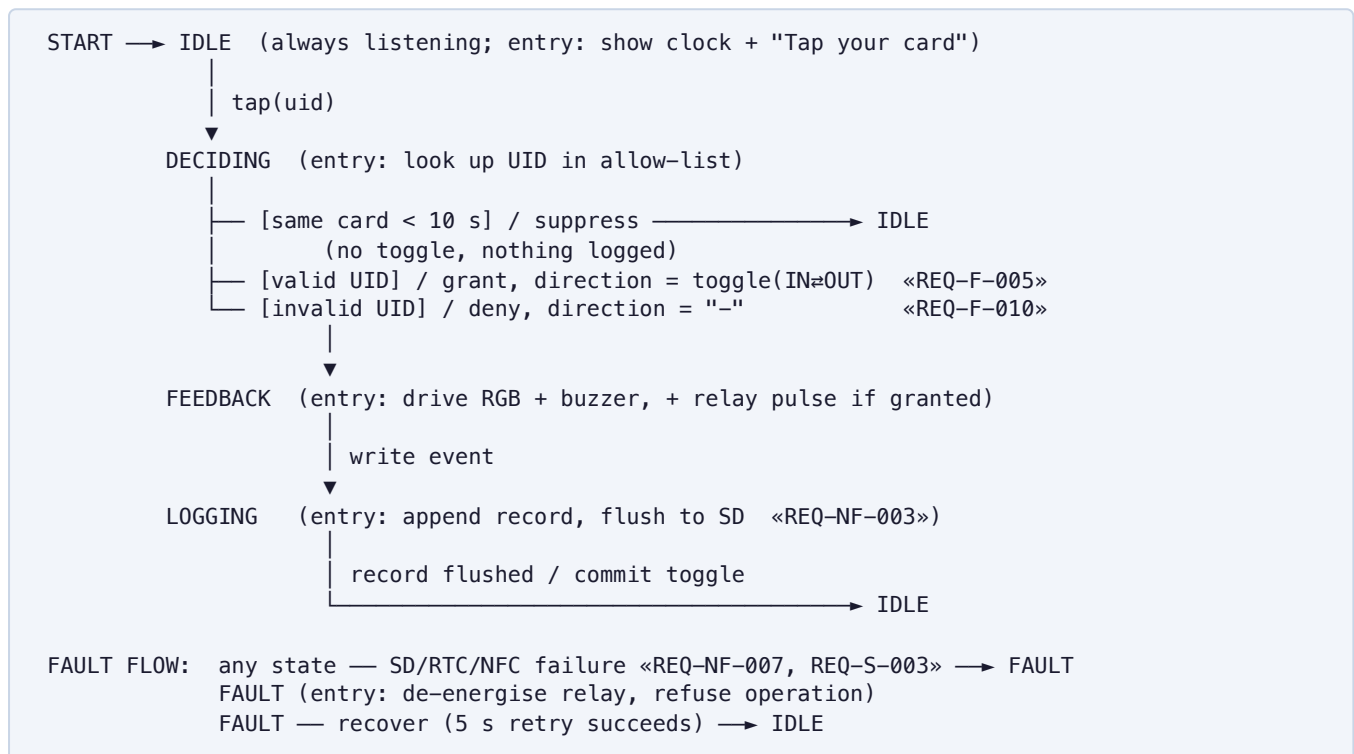
9. Normal operation — auto-toggle flow

1. **Idle (always listening):** OLED shows the RTC time and “Tap your card”; RGB shows the dim idle tint. There is no arming step.
2. **Tap:** present the card. Authorised → green + confirmation tone + relay LED pulse + “GRANTED IN” or “GRANTED OUT” with the person’s name. Unknown → red + error tone + “DENIED” (relay never

actuates, REQ-F-010).

3. **Direction:** each authorised card alternates automatically — first granted tap logs IN, the next OUT, and so on. The current toggle state is rebuilt from `attendance.csv` at start-up, so it survives power cycles. Denied taps log direction `-`.
4. **10 s same-card delay:** after any decided tap, the same card is ignored for 10 s — remove the card after tapping. Other cards are read normally during that window.
5. **Review:** remove the SD card and open `attendance.csv` in any spreadsheet, or edit `allowlist.txt` in any text editor. (The Wi-Fi log-viewing page, REQ-F-011, is an optional stretch feature and is not part of this baseline.)

State machine (per `sen-TapIn.png`, buttons removed)



10. Troubleshooting

Symptom	Likely cause	Fix
OLED blank, nothing happens	OLED init failed — first fault blocks start-up	Check I2C wiring (GP4/GP5), 3.3 V supply; run the I2C scan (§8.1). Watch the REPL: FAULT: ... lines are printed there too.
OLED shows “FAULT / NFC not found”	DFR0231-H in UART mode, or bus wiring	Set the module’s mode switch to I2C; verify 0x24 appears in the scan.
OLED shows “FAULT / SD card”	Card absent, not FAT32, or SPI wiring	Re-seat card; verify format; check GP16/17/18/19 and the module’s supply (§4.2).
OLED shows “FAULT / RTC not found”	DS3231 wiring / power	Verify 0x68 in the scan; check the coin cell is seated.
Relay clicks ON at boot	Active-low relay board	Set RELAY_ACTIVE_HIGH = False in config.py (§4.3).

RGB colours wrong / red-green swapped	Colour order mismatch	Toggle WS2811_ORDER between "RGB" and "GRB" in config.py.
RGB flickers or stays dark	3.3 V data into 5 V part (REQ-HW-004)	Fit the 74AHCT125 level shifter (§4.6); try WS2811_TIMING = 0 for 400 kHz parts.
Buzzer silent or single fixed note	Active buzzer fitted instead of passive	Use a passive buzzer on GP15.
Card read but always Denied	UID mismatch in allow-list	Copy the UID exactly as logged in attendance.csv (uppercase hex, no separators).
Second tap does nothing	10 s same-card delay active	By design: the same card is ignored for 10 s after a decided tap. Wait out the window (tunable via DUP_SUPPRESS_MS in config.py).
Direction seems wrong (IN when expecting OUT)	Toggle state comes from the log	The next direction is the opposite of the card's last GRANTED record in attendance.csv. A missed tap desynchronises it; the next tap corrects the alternation.

Firmware modules follow CS-003: the pin map lives only in board.py; tunables (timeouts, colours, tones, paths) only in config.py. Record any tuning change per REQ-SPEC-003 §6.